NATIONAL INSTITIUTE OF TECHNOLOGY, PATNA

(An Institute of National Importance under M.H.R.D Government of India)

HOSPITAL MANAGEMENT SYSTEM

SUBMITTED BY:

ROLLNO	NAME
2406065	Aarav Raj
2406072	Surya Kumar Yadav
2406074	Animesh Singh
2406076	Harsh Kumar

INTRODUCTION

LifeCare Hospital Management System is a comprehensive full-stack web application designed to streamline and digitize hospital operations. The system manages patient registrations, doctor consultations, appointment scheduling, prescription tracking, billing, and administrative functions. The project was built to eliminate manual healthcare record management and reduce patient wait times. It solves the critical problem of fragmented patient data, inefficient appointment booking, and poor communication between patients and doctors by providing a centralized, real-time digital platform accessible to both patients and hospital staff.

OBJECTIVES OF THE PROJECT

The primary objectives of the Hospital Management System are:

1. **To Build a Complete Hospital Management System for Patients & Admin**
    a. Create a patient portal for viewing/booking appointments and accessing medical records
    b. Develop an admin dashboard for managing doctors, departments, and patient data
    c. Implement role-based access control for different user types
    d. Enable seamless communication between patients and healthcare providers

2. **To Store and Retrieve Data Using Relational Database (RDBMS)**
    a. Design normalized MySQL database schema with proper relationships
    b. Implement CRUD operations for all entities (patients, doctors, appointments, etc.)
    c. Ensure data integrity through foreign key constraints and indexes
    d. Optimize query performance for real-time data retrieval

3. **To Provide a Modern, User-Friendly Web Interface**
    a. Create responsive UI using React with Vite for fast development
    b. Implement intuitive navigation and real-time updates
    c. Ensure accessibility and professional healthcare-compliant design
    d. Support both desktop and mobile devices

4. **To Ensure Secure and Scalable Architecture**
    a. Implement environment variable-based configuration for security
    b. Use CORS and proper authentication mechanisms
    c. Design scalable API endpoints for future enhancements
    d. Follow best practices for data protection and privacy

SYSTEM REQUIREMENTS

a) Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 18+)
Frontend Framework	React 18+ with Vite
Backend Framework	Node.js 14+ with Express.js
Database	MySQL 5.7+ or MySQL 8.0
Programming Language	JavaScript (ES6+)
IDE/Editor	Visual Studio Code 1.70+
Version Control	Git & GitHub
Package Manager	npm 6+ or yarn

b) Tools & Libraries Used

Frontend:

- React Router (Page navigation)
- Axios (API communication)
- CSS3 (Styling)
- Modern JavaScript (ES6+)

Backend:

- Express.js (Web framework)
- MySQL2 (Database driver)
- CORS (Cross-Origin Resource Sharing)
- dotenv (Environment variables)

Database:

- MySQL Workbench (Database management)

INSTALLATION & CONFIGURATION GUIDE

Step-by-Step Setup

Step 1: Install MySQL

- Download from mysql.com
- Create new user: root / password
- Run: mysql -u root -p

Step 2: Create Database
Sql:
CREATE DATABASE hospital_management;
USE hospital_management;
-- *Run all CREATE TABLE statements provided in System Requirements section*

Step 3: Setup Backend
Bash:
mkdir hospital-backend
cd hospital-backend
npm init -y
npm install express mysql2 cors dotenv

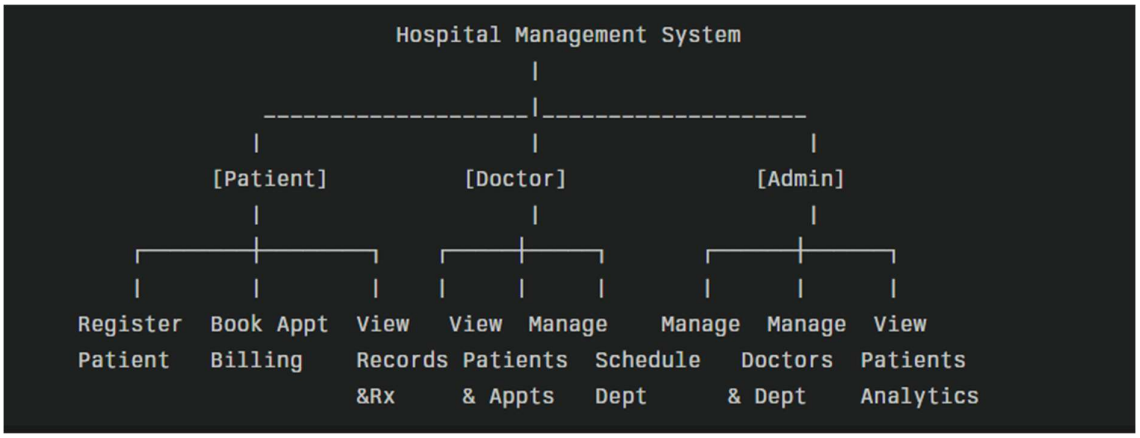
 # Create .env file with credentials
 # Create index.js with all API endpoints
node index.js

Step 4: Setup Frontend
Bash:
npm create vite@latest hospital-frontend -- --template react
cd hospital-frontend
npm install
npm install axios react-router-dom
npm run dev
Testing the System

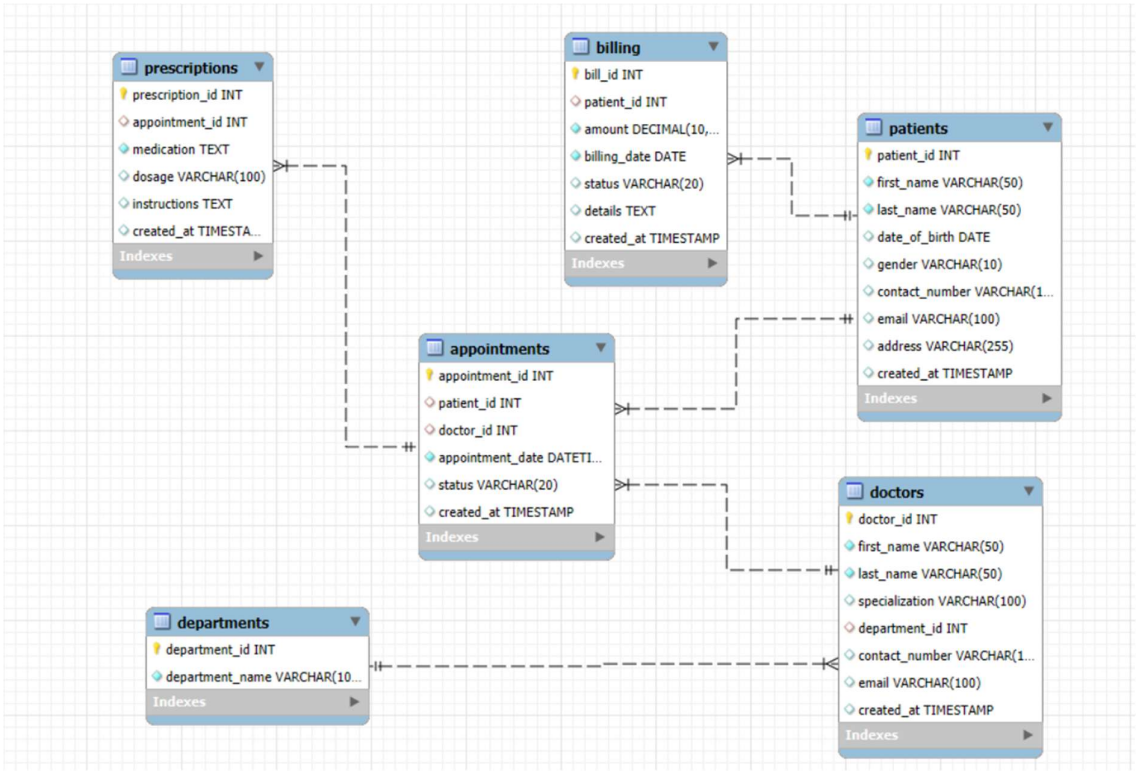
1. Register a patient at: http://localhost:5173/register
2. Copy Patient ID from confirmation
3. Go to Dashboard and login with Patient ID
4. Browse doctors at: http://localhost:5173/doctors
5. Book an appointment from doctor card
6. Check appointment in dashboard

SYSTEM DESIGN

- **Use Case Diagram**



- **Entity Relationship Diagram (ER Diagram)**



METHODOLOGY / WORKING

Program Flow (System Workflow)

Website Navigation

- User visits website: **localhost:5173**
- Available navigation options:
    - **Home** (Information)
    - **Find Doctors** (Browse & Filter)
    - **Register** (Patient Signup)
    - **Dashboard** (Login using Patient ID)
    - **Contact Us** (Support)

Patient Registration Flow

- User fills registration form
- Data sent to backend: **POST /api/patients**
- Backend validates form data
- Stores patient data in **MySQL**
- Generates **Patient ID (AUTO_INCREMENT)**
- Confirmation page displays Patient ID
- User saves Patient ID for future login

Patient Dashboard Login

- User enters **Patient ID**
- Backend queries MySQL for patient details
- Fetches:
    - Appointments
    - Prescriptions
    - Billing details
- Dashboard displays relevant stats and user options
- User can:
    - Book appointments
    - View medical records

Appointment Booking Flow

- Backend fetches list of available doctors
- Patient selects **doctor, date, and time**
- Sends request: **POST /api/appointments**
- Backend saves data in **appointments** table
- Shows success notification
- Appointment appears in patient dashboard

MODULES EXPLANATION

MODULE 1: PATIENT REGISTRATION MODULE

Functionality:

- User creates new account with personal details
- System generates unique Patient ID
- Data validated before storing in database

Configuration & Setup:

Step 1: Database Table Creation

```
CREATE TABLE patients (
   patient_id INT AUTO_INCREMENT PRIMARY KEY,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL,
   date_of_birth DATE,
   gender VARCHAR(10),
   contact_number VARCHAR(15),
   email VARCHAR(100) UNIQUE,
   address VARCHAR(255),
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2: Backend API Configuration

File: hospital-backend/index.js

javascript

```
// POST endpoint for patient registration
app.post('/api/patients', (req, res) => {
  const { first_name, last_name, date_of_birth, gender, contact_number, email, address } = req.body;

  // Validation
  if (!first_name || !last_name || !email || !contact_number) {
    return res.status(400).json({ error: 'Missing required fields' });
  }

  const sql = 'INSERT INTO patients (first_name, last_name, date_of_birth, gender, contact_number, email, address) VALUES (?, ?, ?, ?, ?, ?, ?)';

  db.query(sql, [first_name, last_name, date_of_birth, gender, contact_number, email, address], (err, result) => {
    if (err) {
```

```
      console.error('Error:', err);
      return res.status(500).json({ error: 'Registration failed' });
    }
    res.status(201).json({
      message: 'Patient registered successfully!',
      patient_id: result.insertId
    });
  });
});
```

Step 3: Frontend Component Configuration
File: src/components/PatientRegistration.jsx
javascript

```
// Handle form submission
const handleSubmit = async (e) => {
  e.preventDefault();

  try {
    const response = await axios.post(
      'http://localhost:3001/api/patients',
      formData
    );

    // Show Patient ID to user
    setRegisteredPatient({
      ...formData,
      patient_id: response.data.patient_id
    });
  } catch (error) {
    console.error('Registration error:', error);
    setMessage('Registration failed');
  }
};
```

Step 4: Environment Configuration
File: hospital-backend/.env
text

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=hospital_management
PORT=3001
```

User Manual - Patient Registration:

1. Click "Get Started" or "Register" button on homepage
2. Fill in form fields:
    - First Name (required)
    - Last Name (required)
    - Email (required, must be unique)
    - Date of Birth (required)
    - Gender (optional)
    - Contact Number (required)
    - Address (optional)
3. Click "Register Now" button
4. **Important:** Copy and save your Patient ID from confirmation page
5. Use this ID to login to Patient Dashboard

MODULE 2: PATIENT DASHBOARD LOGIN MODULE

Functionality:

- Patient logs in using unique Patient ID
- Access personal health records
- View and manage appointments
- Check prescriptions and billing

Configuration & Setup:

Step 1: Database Queries Setup

File: hospital-backend/index.js

javascript

```
// GET patient profile
app.get('/api/patients/:patientId', (req, res) => {
  const sql = 'SELECT * FROM patients WHERE patient_id = ?';
  db.query(sql, [req.params.patientId], (err, results) => {
    if (err || results.length === 0) {
      return res.status(404).json({ error: 'Patient not found' });
    }
    res.json(results);
  });
});

// GET patient appointments
app.get('/api/patients/:patientId/appointments', (req, res) => {
  const sql = `
    SELECT a.*, d.first_name, d.last_name, d.specialization
    FROM appointments a
    JOIN doctors d ON a.doctor_id = d.doctor_id
    WHERE a.patient_id = ?
```

```
    ORDER BY a.appointment_date DESC`;

  db.query(sql, [req.params.patientId], (err, results) => {
    if (err) return res.status(500).json({ error: 'Error fetching appointments' });
    res.json(results || []);
  });
});

// GET prescriptions
app.get('/api/patients/:patientId/prescriptions', (req, res) => {
  const sql = `
    SELECT p.* FROM prescriptions p
    JOIN appointments a ON p.appointment_id = a.appointment_id
    WHERE a.patient_id = ?
    ORDER BY p.created_at DESC`;

  db.query(sql, [req.params.patientId], (err, results) => {
    if (err) return res.status(500).json({ error: 'Error fetching prescriptions' });
    res.json(results || []);
  });
});

// GET billing records
app.get('/api/patients/:patientId/billing', (req, res) => {
  const sql = 'SELECT * FROM billing WHERE patient_id = ? ORDER BY billing_date
DESC';
  db.query(sql, [req.params.patientId], (err, results) => {
    if (err) return res.status(500).json({ error: 'Error fetching billing' });
    res.json(results || []);
  });
});
```

Step 2: Frontend Dashboard Setup

File: src/pages/PatientDashboard.jsx

javascript

```
// Login handler
const handleLogin = async (e) => {
  e.preventDefault();

  try {
    // Make 4 simultaneous requests
    const [patientRes, appointmentsRes, prescriptionsRes, billingRes] = await Promise.all([
      axios.get(`http://localhost:3001/api/patients/${patientId}`),
      axios.get(`http://localhost:3001/api/patients/${patientId}/appointments`),
```

```
    axios.get(`http://localhost:3001/api/patients/${patientId}/prescriptions`),
    axios.get(`http://localhost:3001/api/patients/${patientId}/billing`)
   ]);

   // Set state with retrieved data
   setPatient(patientRes.data);
   setAppointments(appointmentsRes.data);
   setPrescriptions(prescriptionsRes.data);
   setBilling(billingRes.data);
   setAuthenticated(true);

  } catch (error) {
   setError('Invalid Patient ID or connection error');
  }
};
```

User Manual - Patient Dashboard Login:

1. Click "Dashboard" in navigation
2. Enter your **Patient ID** (received during registration)
3. Click "Access Portal" button
4. Dashboard shows:
    - Welcome message with your name
    - 4 stat cards (Upcoming Appointments, Prescriptions, Bills, Amount Due)
    - Tabs: Overview | Appointments | Prescriptions | Billing
5. **Overview Tab:**
    - View your profile information
    - Quick action buttons
    - Recent appointments preview
6. **Appointments Tab:**
    - View all appointments with doctor details
    - Click "Details" to see full information
    - Click "Cancel" to cancel upcoming appointments
    - Click "+ Book New Appointment" to schedule
7. **Prescriptions Tab:**
    - View all prescribed medications
    - See dosage and instructions
    - Download prescriptions as needed
8. **Billing Tab:**
    - View payment history
    - Check pending bills
    - Click "Pay Now" for unpaid bills

MODULE 3: DOCTOR BROWSING & APPOINTMENT BOOKING MODULE

Functionality:

- Browse all available doctors
- Filter by specialization
- View doctor details and contact info
- Book appointments directly

Configuration & Setup:

Step 1: Database Setup for Doctors

sql

```
-- Create departments
INSERT INTO departments (department_name) VALUES
('Cardiology'), ('Orthopedics'), ('Neurology'), ('Pediatrics');

-- Create doctors
INSERT INTO doctors (first_name, last_name, specialization, department_id, contact_number, email) VALUES
('Rajesh', 'Sharma', 'Cardiologist', 1, '9876543210', 'rajesh@lifecare.com'),
('Priya', 'Verma', 'Orthopedic Surgeon', 2, '9876543211', 'priya@lifecare.com'),
('Vikram', 'Patel', 'Neurologist', 3, '9876543212', 'vikram@lifecare.com'),
('Anjali', 'Desai', 'Pediatrician', 4, '9876543213', 'anjali@lifecare.com');
```

Step 2: Backend API for Doctors

javascript

```
// GET all doctors with details
app.get('/api/doctors', (req, res) => {
  const sql = `
    SELECT d.*, dept.department_name, COUNT(a.appointment_id) as total_appointments
    FROM doctors d
    LEFT JOIN departments dept ON d.department_id = dept.department_id
    LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
    GROUP BY d.doctor_id
    ORDER BY d.first_name ASC`;

  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error: 'Error fetching doctors' });
    res.json(results);
  });
});

// POST new appointment
app.post('/api/appointments', (req, res) => {
```

```
  const { patient_id, doctor_id, appointment_date } = req.body;

  const sql = 'INSERT INTO appointments (patient_id, doctor_id, appointment_date, status)
VALUES (?, ?, ?, "Scheduled")';

  db.query(sql, [patient_id, doctor_id, appointment_date], (err, result) => {
    if (err) return res.status(500).json({ error: 'Booking failed' });
    res.status(201).json({
      message: 'Appointment booked successfully!',
      appointment_id: result.insertId
    });
  });
});
```

Step 3: Frontend Components

File: src/pages/DoctorsPage.jsx

- Display all doctors in grid/list view
- Filter by specialization
- Search by doctor name
- Click doctor card to see full details

File: src/components/DoctorCard.jsx

- Show doctor avatar, name, specialization
- Display ratings and patient count
- Contact button for phone/email
- Book appointment button

Step 4: Appointment Booking Modal

File: src/components/BookAppointmentModal.jsx

javascript

```
const handleBooking = async (e) => {
  e.preventDefault();

  try {
    const response = await axios.post(
      'http://localhost:3001/api/appointments',
      {
        patient_id: patientId,
        doctor_id: selectedDoctor,
        appointment_date: appointmentDate
      }
    );

    setMessage('✓ Appointment booked successfully!');
```

```
    // Refresh appointments list
    setTimeout(() => onSuccess(), 2000);

  } catch (error) {
    setMessage('Booking failed');
  }
};
```

User Manual - Doctors Page & Booking:

1. Click "Find Doctors" in navigation
2. **Browse Doctors:**
    - View all doctors in professional cards
    - See specialization, experience, ratings
    - Scroll through available doctors
3. **Filter & Search:**
    - Use search box to find by name
    - Select specialty from dropdown
    - Toggle between Grid/List view
4. **View Doctor Details:**
    - Click doctor card to see full profile
    - View phone number and email
    - See department and consultation count
5. **Book Appointment:**
    - Click "Book Now" button
    - Modal opens with booking form
    - Enter your **Patient ID**
    - Select appointment date & time
    - Click "Book Appointment"
    - Confirmation message with Appointment ID
6. **Contact Doctor:**
    - Click "Contact" for direct contact
    - View email and phone number
    - Can call or email directly

CONCLUSION

The Hospital Management System successfully demonstrates full-stack web development using modern technologies. It implements OOP principles through modular React components, manages complex relational data in MySQL, and provides a scalable backend architecture. The system improves healthcare delivery by enabling digital patient registration, appointment scheduling, and medical record management. Future enhancements could include payment gateway integration, SMS notifications, and analytics dashboard for hospital administration.